TTLSTT~

TOTALISATION

Réconciliation
avec le produit logiciel

Partie A

Léo Michel Bélanger

Publié par ORIGINALIUM | TTLSTT~

© Léo Michel Bélanger - Tous droits réservés
Images et couverture par Soléane Bélanger
Publié par ORIGINALIUM | TTLSTT~

TTLSTT~ - Réconciliation avec le produit logiciel - Partie A

Série ORIGINALIUM

© Léo Michel Bélanger - Tous droits réservés
Images et couverture par Soléane Bélanger
Publié par ORIGINALIUM | TTLSTT~

Aucune partie de cet ouvrage ne peut être reproduite, transmise ou distribuée sous quelque forme ou par quelque moyen que ce soit - électronique, mécanique, photocopie, enregistrement ou autre - sans l'autorisation écrite préalable de l'auteur, sauf dans le cadre de brèves citations utilisées dans des comptes rendus ou articles critiques.

Ce livre fait partie de la série ORIGINALIUM. Il constitue le deuxième volume de la collection ŢŢĻŞŢ~, suite directe de *L1 - L'origine de l'art de l'excellence*.

TOTALISATION - Ţ=TO, Ţ=TA, Ļ=LI, Ş=SA, Ţ=TI, ~=ON

Imprimé et publié par ORIGINALIUM | ŢŢĻŞŢ~
Montréal, Québec, Canada

Table des matières

Préambule - Le produit logiciel mal aimé

Le paradoxe fondateur

Pourquoi la réconciliation est nécessaire

Ce que ŢŢĻŞŢ~ apporte de nouveau

Chapitre 1 - Qu'est-ce vraiment qu'un produit logiciel ?

La définition traditionnelle et ses angles morts

Le logiciel comme entité vivante et évolutive

Les dimensions oubliées du produit

Le produit vu par Totalisation

Chapitre 2 - La nature unique du logiciel comme produit

Ce qui distingue le logiciel de tout autre produit humain

L'immatérialité et ses conséquences

La complexité invisible

Le paradoxe de la perfection

ŢŢĻŞŢ~ fait la paix avec cette nature

Chapitre 3 - Les fractures dans la relation avec le produit logiciel

La fracture entre développeurs et utilisateurs

La fracture entre vision et réalité

La fracture entre qualité perçue et qualité réelle

La fracture temporelle

Comment ces fractures gèrent des logiciels décevants

Chapitre 4 - ŢŢĻŞŢ~ face au produit : une nouvelle ontologie

Le produit comme tout indivisible

La réconciliation avec les exigences

La réconciliation avec le code

La réconciliation avec les tests

La réconciliation avec la documentation

Note de fin de Partie A - Vers la Partie B

« On ne comprend pas ce qu'on construit.

On construit ce qu'on ne comprendra jamais tout à fait.

Et pourtant, on continue - parce que c'est dans cet écart

entre la compréhension et la création

que réside l'essence même du logiciel. »

- Léo Michel Bélanger

ORIGINALIUM | ŢŢĻŞŢ~

Totalisation - Deux dimensions

Dimension 1 - Substantif linguistique : « Action d'additionner ; action de considérer l'ensemble, le total, de synthétiser. »

Dimension 2 - Philosophie ORIGINALIUM : « Rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption. »

Préambule

Le produit logiciel mal aimé

Ce préambule pose la question fondamentale qui traverse tout ce livre : pourquoi l'industrie qui crée le logiciel semble-t-elle si souvent ne pas le comprendre - et qu'est-ce que Totalisation peut y faire ?

Le paradoxe fondateur

Il y a quelque chose de profondément étrange dans notre relation au produit logiciel. On en construit des centaines de milliers chaque année. Des équipes entiers y consacrent leur carrère, leur intelligence, leur passion. On investit des milliards de dollars dans leur conception, leur développement, leur mise en marché. Et pourtant - et c'est là où le paradoxe se révèle dans toute son étrangeté - on ne les comprend pas vraiment. Pas dans le sens profond du mot « comprendre ». Pas dans le sens où la compréhension engendre le respect, l'intention claire, la vision cohérente.

On sait *comment* coder un logiciel. On sait *comment* le livrer. On sait *comment* le vendre. Mais on ne sait pas toujours, avec clarté et conviction, *ce qu'est* un produit logiciel dans sa totalité. On ne sait pas toujours ce qu'on crée vraiment quand on crée un logiciel - non pas au sens technique, mais au sens ontologique, au sens du « qu'est-ce que c'est ? » fondamental qui devrait précéder toute création sérieuse.

Ce paradoxe n'est pas une critique. Ce n'est pas un procès contre les développeurs, les gestionnaires de produit, les architectes ou les concepteurs. C'est simplement un constat : notre industrie a grandi si vite, a accéléré si furieusement, qu'elle n'a pas eu le luxe de s'arrêter pour se demander : « Mais qu'est-ce qu'on fait, au fond ? » Elle a couru avant de marcher. Elle a livré avant de comprendre. Elle a scaling avant de savoir ce qu'elle amplifiait.

Le résultat de ce paradoxe, tu le connais. Tu le vis peut-être chaque jour. Ce sont des logiciels qui naissent orphelins de vision. Des produits qu'on développe en compartiments étanches où chaque spécialiste ne voit que sa tranche. Des projets qui dérivent, se gonflent, se complexifient sans qu'on sache vraiment vers quoi ils tendent. Des livrables qui arrivent « terminés » mais que personne ne reconnatît comme étant ce qu'on avait imaginé.

C'est ce paradoxe que ce livre veut dénouer. Pas en t'offrant une méthode de plus. Pas en ajoutant une couche de processus par-dessus des processus déjà épais. Mais en revenant à la source : *qu'est-ce qu'un produit logiciel ?* Et comment ŢŢĻṢŢ~ - Totalisation - nous permet de le voir autrement, de le vivre autrement, de le construire autrement.

Pourquoi la réconciliation est nécessaire

Le mot « réconciliation » est intentionnel. Il suppose une brouille. Une distance. Une relation abîmée. Et c'est exactement ce qu'il s'est passé : l'industrie du logiciel s'est brouillée avec son propre produit. Pas de façon spectaculaire. Pas en un seul instant. Mais progressivement, insidieusement, au fil des méthodologies qui se sont enserrantes autour du produit comme du lièrre autour d'un mur - jusqu'à ce que ce soit le lièrre qu'on voit, et non plus le mur.

La réconciliation est nécessaire parce qu'on ne peut pas bien construire ce qu'on ne comprend pas. On ne peut pas aimer ce dont on a peur. On ne peut pas prendre soin de ce qu'on méprise ou que l'on instrumentalise. Et trop souvent, le produit logiciel a été maltraité - non par malveillance, mais par ignorance. Par ignorance de sa nature. Par ignorance de ce qu'il demande à ceux qui le créent. Par ignorance de ce qu'il représente vraiment dans la vie de ceux qui l'utilisent.

Se réconcilier avec le produit logiciel, c'est accepter de le regarder en face, sans les filtres rassurants des méthodologies, des frameworks ou des schémas d'architecture. C'est le voir tel qu'il est : complexe, vivant, imparfait, en perpétuelle évolution, porteur de sens, porteur de valeur, porteur de la trace de ceux qui l'ont créé et de ceux qui l'utilisent. C'est renouer avec lui à partir de la vérité, pas à partir des illusions que l'on entretient souvent à son sujet.

Cette réconciliation exige du courage. Elle demande qu'on admette qu'on s'est peut-être trompé de lunettes. Qu'on a parfois confondu le contenant et le contenu. Qu'on a pris le ticket pour le voyage. Qu'on a optimisé des parties sans avoir jamais vu le tout. C'est un acte d'humilité intellectuelle autant qu'une ouverture philosophique.

Ce que T̰T̰L̰S̰T̰~ apporte de nouveau

Dans *L1 - L'origine de l'art de l'excellence*, les bases de Totalisation ont été posées. Tu as découvert ce que signifie *rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption.* Tu as compris que Totalisation n'est pas une technique mais un état d'esprit, une façon d'être face à la création.

Ce deuxième livre applique cette philosophie à l'objet même qui définit notre ère : le produit logiciel. Il explore ce que Totalisation voit quand elle regarde un logiciel - pas seulement ses fonctionnalités, pas seulement son code, pas seulement ses utilisateurs, mais sa totalité : son avant, son pendant, son après. Sa partie visible et sa partie invisible. Sa dimension technique et sa dimension humaine. Son présent et son devenir.

Ce que T̰T̰L̰S̰T̰~ apporte de nouveau, c'est une *ontologie du produit logiciel*. Une façon de le définir non pas par ce qu'il fait, mais par ce qu'il *est*. Non pas par ses fonctions, mais par sa nature. Cette ontologie permet de dépasser les débats techniques pour atteindre quelque chose de plus fondamental : une compréhension intégrale qui réconcilie toutes les parties prenantes autour d'une vision commune et véritable.

La méthode BEASTEST - ce cadre d'excellence à sept niveaux qui traverse toute la philosophie ORIGINALIUM - sera convoquée ici non comme une grille

de réévaluation ou d'audit, mais comme une lens, une loupe philosophique permettant de voir le produit logiciel dans ses différentes dimensions de maturité. Du ROUGE à la survie brute, jusqu'au VIOLET où le produit attaint une cohérence qui dépasse la somme de ses parties.

La Méthode BEASTEST - Les 7 Niveaux

La méthode BEASTEST constitue le cadre d'excellence de la philosophie ORIGINALIUM. Elle propose sept niveaux de maturité - ROUGE, ORANGE, JAUNE, VERT, BLEU, INDIGO, VIOLET - qui ne sont pas des échelons à gravir mécaniquement, mais des modes d'être face à la création. Chaque niveau représente une qualité de présence, d'intention et de compréhension vis-à-vis du produit logiciel. Ce cadre sera exploré dans sa relation au produit tout au long des chapitres qui suivent.

Chapitre 1

Qu'est-ce vraiment qu'un produit logiciel ?

Avant de pouvoir réconcilier quoi que ce soit, il faut nommer ce dont on parle. Ce chapitre désinstalle les définitions paresseuses pour installer quelque chose de plus juste, de plus complet - quelque chose de totalisé.

La définition traditionnelle et ses angles morts

Si tu demandes à dix professionnels du logiciel de définir ce qu'est un produit logiciel, tu obtiendras probablement dix variantes de la même réponse. Un produit logiciel est, dira-t-on, un ensemble de programmes, de données et de documentation qui fonctionne selon des spécifications définies, livré à des utilisateurs pour satisfaire des besoins identifiés. C'est fonctionnel. C'est précis. C'est même correct, dans un certain sens.

Mais cette définition a des angles morts énormes. Elle parle de « programmes » comme si le code était la chose elle-même, et non pas la trace d'une intention. Elle parle de « spécifications définies » comme si les spécifications étaient fermées, stables, finales - alors qu'on sait tous qu'elles ne le sont jamais vraiment. Elle parle de « besoins identifiés » comme si les besoins étaient des entités fixes qui attendaient sagement qu'on vienne les identifier, alors qu'ils évoluent, se transforment, se contredisent et parfois même se découvrent en cours de développement.

La définition traditionnelle est une définition d'ingénieur : rigoureuse dans son périmètre, aveugle en dehors de lui. Elle capture la *structure* du produit mais pas sa *nature*. Elle décrit ce que le produit contient sans décrire ce que le produit *est*. Elle nomme les composantes sans nommer l'essence. Et c'est de cette lacune que naît une grande partie des problèmes que l'industrie du logiciel rencontre depuis des décennies.

L'angle mort le plus fondamental de la définition traditionnelle est son atemporalité. Elle photographie le produit à un instant t. Elle ne voit pas son avant - tout ce qui a précédé sa naissance, toutes les décisions, les compromis, les visions et les renoncements qui l'ont faonné. Elle ne voit pas son après - tout ce qu'il deviendra, tout ce qu'il entraînera, tous les usages qu'on n'avait pas prévus et qui redessineront sa signification. Elle ne voit que

son « pendant » - ce qu'il fait maintenant, ce qu'il fait fonctionner, ce qu'il livre aujourd'hui.

Totalisation, elle, refuse cette temporalité tronquée. Elle refuse de voir le produit logiciel comme une photographie et insiste pour le voir comme un film - un film qui a un avant, un pendant et un après, et où tous les plans sont nécessaires à la compréhension du tout.

Le logiciel comme entité vivante et évolutive

Il y a une métaphore que les gens du logiciel utilisent souvent, parfois sans réaliser à quel point elle est vraie : ils disent que leur code est « vivant ». Ils parlent de « la vie d'un projet », du « cycle de vie d'un produit ». Ce vocabulaire de la vie n'est pas anodin. Il trahit une intuition profonde que la définition technique ne capte pas : un produit logiciel se comporte comme un organisme vivant, pas comme un objet inerte.

Un objet inerte - une chaise, un pont, un four - est fini quand il est terminé. Sa nature ne change pas. Ses propriétés sont stables. Il vieillit, il s'use, mais il reste fondamentalement le même dans sa définition. Un produit logiciel, lui, n'est jamais vraiment fini. Il mue. Il se transforme. Il absorbe les retours de ses utilisateurs comme un organisme absorbe des nutriments, et il en sort différent. Il répond aux pressions de son environnement - technologique, réglementaire, concurrentiel - comme un être vivant répond aux pressions de son écosystème.

Cette nature vivante du logiciel a une conséquence philosophique majeure : on ne peut pas le traiter comme un objet à fabriquer une fois pour toutes. On doit le traiter comme une relation à entretenir. Et les relations demandent de l'attention, de la présence, de l'adaptation. Elles demandent qu'on soit là -

avant, pendant et après - en continu, sans interruption. Elles demandent, en un mot, de la Totalisation.

Reconnatître le logiciel comme entité vivante, c'est accepter qu'il a des besoins qui évoluent. Qu'il a une histoire qui le marque. Qu'il a un contexte qui le définit autant que son code. Qu'il a une identité qui se construit dans le temps, pas seulement dans les spécifications initiales. C'est accepter, enfin, que s'en occuper demande une intelligence qui va bien au-delà de la compétence technique : une intelligence du soin, de la continuité, de la cohérence dans la durée.

Les dimensions oubliées du produit

La définition traditionnelle retient trois dimensions du produit logiciel : le code, les données, la documentation. Mais un produit logiciel existe dans bien d'autres dimensions que ces trois-là. Des dimensions que l'industrie a tendance à négliger parce qu'elles sont moins facilement quantifiables, moins directement facturables, moins susceptibles d'apparaître dans un sprint backlog.

La dimension humaine : Chaque produit logiciel porte en lui la trace de ceux qui l'ont créé. Leurs décisions, leurs valeurs, leurs angles morts, leurs génialités, leurs peurs. Un code n'est pas neutre. Il est le reflet d'une culture, d'une époque, d'un collectif humain avec ses forces et ses contradictions. Quand on ignore la dimension humaine du produit, on se prive de comprendre pourquoi il est ce qu'il est - et donc de pouvoir vraiment l'améliorer.

La dimension temporelle : Un produit logiciel n'existe pas dans l'instant. Il a un passé - les décisions d'architecture prises il y a cinq ans qui contraignent les possibilités d'aujourd'hui. Il a un présent - les fonctionnalités actives, les bugs

ouverts, les utilisateurs connectés en ce moment. Il a un futur - les évolutions anticipées, les tendances technologiques qui le rendront obsolescent, les besoins émergents qu'il devra un jour absorber. Ignorer la temporalité du produit, c'est conduire en ne regardant que le compteur de vitesse - sans voir la route, sans voir le rétroviseur, sans voir le paysage qui change.

La dimension contextuelle : Un produit logiciel n'existe jamais dans le vide. Il existe dans un écosystème : un marché, une industrie, un cadre réglementaire, une culture d'utilisation, des systèmes adjacents. Sa valeur n'est pas intrinsèque - elle est relationnelle. Ce qu'il vaut dépend de ce qui l'entoure. Ce qu'il peut faire dépend de ce avec quoi il peut interagir. L'ignorer, c'est comme évaluer la valeur d'un mot hors de sa phrase.

Ces trois dimensions - humaine, temporelle, contextuelle - sont les dimensions oubliées du produit logiciel. Ce ne sont pas des « soft factors » qu'on peut mettre de côté sans conséquences. Ce sont des constituantes essentielles de ce qu'est le produit. Les négliger, c'est travailler sur une image appauvrie du réel - et s'étonner ensuite que les résultats soient décevants.

Le produit vu par Totalisation : un ensemble cohérent avant, pendant et après

Totalisation propose une définition du produit logiciel qui intègre toutes ces dimensions. Pour ŢŢĻŞŢ~, un produit logiciel est : *un ensemble cohérent d'intentions, de décisions, de créations et de relations, qui existe dans la continuité du temps et qui trouve sa complétude dans la totalité de ses dimensions - technique, humaine, temporelle et contextuelle.*

Cette définition change tout. Elle ne parle pas de « programmes » ou de « spécifications ». Elle parle d'*intentions* - parce qu'un produit logiciel

commence toujours par une intention, une vision, un « pourquoi » qui devrait rester central tout au long de sa vie. Elle parle de *décisions* - parce qu'un produit est fondamentalement une suite de décisions, grandes et petites, qui s'accumulent et se solidifient en architecture, en interface, en expérience. Elle parle de *créations* - tout ce qui est matériellement produit, code, tests, documentation, design. Et elle parle de *relations* - parce qu'un produit qui n'existe pas dans la relation avec ses utilisateurs n'est qu'un artefact sans vie.

La Totalisation vu le produit dans son ensemble cohérent. Pas en parties. Pas en « frontend » et « backend », pas en « UX » et « technique », pas en « ce qu'on a livré » et « ce qu'on n'a pas eu le temps de faire ». Le produit est un tout. Et c'est ce tout que ŢŢĻŞŢ~ s'engage à rassembler et à réaliser - avant, pendant et après - en continu, sans interruption.

> *Le produit logiciel n'est pas ce que tu livres. C'est tout ce qui constitue ce que tu livres - y compris ce que tu as failli ne pas livrer, ce que tu as choisi de ne pas livrer, et ce que la prochaine version devra réparer. La totalité est là, toujours. La question est : est-ce que tu la vois ?*

Chapitre 2

La nature unique du logiciel comme produit

Le logiciel est un produit radicalement différent de tout ce que l'humanité a créé avant lui. Ce chapitre explore ce qui le rend unique - et pourquoi cette unicité exige une philosophie spécifique pour être comprise et honorée.

Ce qui distingue le logiciel de tout autre produit humain

L'humanité crée des produits depuis des millénaires. Des outils de pierre, des ponts, des cathédrales, des véhicules, des médicaments, des oeuvres d'art. Chaque époque a ses artefacts signature, ses créations qui la définissent et la dépassent. Notre époque, c'est le logiciel. Et le logiciel, à bien des égards, est fondamentalement différent de tout ce qui l'a précédé.

Un pont en acier occupe un espace physique. On peut le toucher, le mesurer, le photographier depuis l'autre rive. Un médicament a une formule moléculaire, une masse, une solubility. Une oeuvre d'art a des dimensions, des matériaux, une texture. Tous ces produits existent dans le monde matériel. Leurs propriétés physiques contraignent et définissent ce qu'ils peuvent être et faire.

Le logiciel, lui, n'existe nulle part dans le monde matériel. Il existe dans le monde des idées exécutées. C'est de la pensée organisée de manière à ce qu'une machine puisse la réaliser. Cette nature fondamentalement immatérielle le rend unique, étrange, et extraordinairement difficile à appréhender pour des esprits habitant un monde physique.

Mais l'immatérialité n'est qu'une des dimensions qui distinguent le logiciel. L'autre, c'est sa *plasticité radicale*. On peut modifier un logiciel à moindre coût physique - pas de matériaux à réapprovisionner, pas de chaotîne de montage à reconfigurer, pas d'outillage à remplacer. En théorie, modifier du code ne coûte que du temps et de l'intelligence. En pratique, cette plasticité apparente

cache une rigidité structurelle terrible : la dette technique, l'architecture dégradée, le code spaghetti qui résiste à toute modification comme du béton armé.

Et puis il y a la troisième distinction : le logiciel est le seul produit humain dont la *complexité n'a pas de limite naturelle*. Un pont a des contraintes physiques qui bornent sa complexité. Un médicament a des limites moléculaires. Mais un logiciel peut être infiniment complexe. On peut toujours ajouter une fonctionnalité de plus, une intégration de plus, un cas particulier de plus. Cette infinité potentielle est à la fois sa grandeur et son piège le plus redoutable.

L'immatérialité et ses conséquences sur la qualité

L'immatérialité du logiciel a des conséquences profondes sur la manière dont on comprend - ou ne comprend pas - sa qualité. Quand un pont s'effondre, le problème est immédiatement visible et localisable. Quand un tissu est de mauvaise qualité, les fils lâchent, la couleur part, la texture se dégrade. La qualité matérielle se manifeste dans le monde matériel, de manière visible, palpable, incontestable.

La qualité d'un logiciel, elle, est largement invisible. On ne voit pas si le code est bien structuré ou chaotique. On ne voit pas si l'architecture est solide ou fragile. On ne voit pas si les tests couvrent vraiment les cas critiques ou seulement les cas faciles. On ne voit pas la dette technique qui s'accumule dans les entrailles du système. On voit l'interface. On voit ce que le logiciel fait - quand il le fait. On ne voit pas ce qu'il ne fait pas, ce qu'il risque de ne plus faire, ce qu'il aurait dû faire.

Cette invisibilité de la qualité logicielle crée une asymétrie dévastatrice : ceux qui paient pour le logiciel ne peuvent pas voir ce qu'ils paient. Ceux qui le

créent peuvent donc, consciemment ou non, sacrifier la qualité interne pour la qualité apparente - rendre l'interface belle en laissant l'architecture se dégrader, livrer vite en accumulant de la dette qui finira par coûter bien plus cher à rembourser.

Totalisation répond à ce défi en affirmant qu'on ne peut pas séparer la qualité visible de la qualité invisible. Le tout ne peut pas être excellent si ses parties cachées sont défaillantes. Un logiciel qui paraît bien fonctionner mais repose sur une architecture catastrophique n'est pas un logiciel de qualité : c'est un logiciel qui emprunte à son propre futur pour paraître présentable aujourd'hui. ŢŢĻŞŢ~ refuse cet emprunt. Elle exige la cohérence entre l'apparent et le réel, entre la surface et la profondeur.

La complexité invisible : pourquoi le logiciel résiste à la simplicité

Il existe un mythe persistant dans l'industrie du logiciel : celui de la simplicité comme vertu absolue. « Keep it simple. » « YAGNI - You Aren't Gonna Need It. » « Un bon ingénieur, c'est un ingénieur qui simplifie. » Ces mantras ne sont pas faux. Mais ils masquent une vérité que Totalisation tient à nommer clairement : *le logiciel résiste fondamentalement à la simplicité - non par défaut des ingénieurs, mais par nature de ce qu'il fait.*

Un logiciel modéle la réalité. Et la réalité est complexe. Quand on construit un système de gestion d'inventaire, on essaie de capturer dans du code la complexité réelle des entrepots, des fournisseurs, des commandes, des exceptions, des règles métier qui se contredisent, des cas particuliers qui ont une histoire humaine. Cette complexité n'est pas dans le code : elle est dans le réel. Le code ne fait que la réfléchir. Et on ne peut pas simplifier le reflet sans distorsionner ce qu'il reflète.

La complexité du logiciel est donc en grande partie une complexité *accidentelle* - celle qu'on ajoute par mauvaise architecture, par dette technique, par décisions précipitées - mais aussi une complexité *essentielle* - celle qui vient de la complexité intrinsèque du domaine qu'il modélise. La complexité accidentelle, on peut et on doit la réduire. La complexité essentielle, elle, ne se réduit pas. Elle se gère. Elle se navigue. Elle s'honore par la sophistication de la pensée qu'on lui consacre.

Totalisation fait la distinction entre ces deux formes de complexité. Elle invite à ne pas confondre simplifier là où c'est possible avec nier la complexité là où elle est nécessaire. Elle appelle à avoir l'intelligence de distinguer ce qu'on peut réduire de ce qu'on doit assumer - et à avoir la sérénité d'accepter cette distinction sans la fuir dans des slogans réducteurs.

Le paradoxe de la perfection : toujours fini, jamais terminé

Il existe une tension fondamentale au coeur de tout produit logiciel, une tension que tout développeur a vécu mais que peu ont nommée clairement : le logiciel est toujours « fini » à un moment donné - on le livre, on le déploie, on passe à la version suivante - mais il n'est jamais vraiment « terminé ». Il y a toujours un bug de plus à corriger. Une fonctionnalité de plus à ajouter. Une optimisation de plus à faire. Une ergonomie à améliorer. Un cas particulier à gérer.

Ce paradoxe est source d'une grande souffrance dans l'industrie du logiciel. Les équipes se sentent coupables de livrer quelque chose d'imparfait. Les gestionnaires s'impatientent devant des produits qui ne semblent jamais « done ». Les utilisateurs sont frustrés par des logiciels qui ont des bugs, des lacunes, des incoheréences. Tout le monde souffre du même malaise : l'idéal de la perfection se heurte à la réalité de l'incomplétude.

Totalisation propose une façon de vivre avec ce paradoxe plutôt que de le subir. Elle distingue la *complétude momentanée* - le produit est complet par rapport à ce qu'on a décidé de créer pour cette étape, pour ce contexte, pour ces contraintes - de la *perfection absolue*, qui est une chimere. La complétude momentanée est honorable. Elle est même nécessaire, parce qu'un produit livré et utilisé apprend et évolue, alors qu'un produit parfait qui n'existe pas encore ne sert personne.

L'état VIOLET de la méthode BEASTEST n'est pas un état de perfection absolue. C'est un état de cohérence totale dans l'intention : on sait pourquoi on a fait ce qu'on a fait, on comprend ce qu'on n'a pas encore fait et pourquoi, on a une vision claire de ce que le produit deviendra. La perfection n'est pas dans l'absence de défauts. Elle est dans la clarté de l'intention et dans la totalisation de la vision.

Comment ŢŢĻŞŢ~ fait la paix avec cette nature unique

Faire la paix avec la nature unique du logiciel, c'est d'abord cesser de vouloir qu'il soit ce qu'il n'est pas. Ce n'est pas un produit manufacturier. Ce n'est pas une oeuvre d'art fermée. Ce n'est pas un service immatériel sans forme. C'est *sa propre catégorie de réalité*, avec ses propres lois, ses propres rythmes, ses propres exigences de qualité.

Totalisation fait la paix avec cette nature en adoptant une posture fondamentale : l'*acceptation active*. Accepter que le logiciel soit immatériel ne signifie pas ignorer ses dimensions invisibles - cela signifie en faire une priorité, les rendre visibles par des pratiques délibérées. Accepter que le logiciel soit complexe ne signifie pas capituler devant la complexité - cela signifie l'affronter avec la totalité de son intelligence, sans raccourcis, sans dnéi. Accepter que le logiciel ne soit jamais terminé ne signifie pas se résigner

à livrer n'importe quoi - cela signifie définir avec précision ce que « fini » signifie à chaque étape, et s'y tenir.

Cette paix n'est pas la paix de la résignation. C'est la paix du comprendre. La paix de celui qui cesse de se battre contre la nature de la chose pour commencer à travailler avec elle. Et c'est précisément là que la philosophie ORIGINALIUM trouve sa puissance : dans cette intelligence de la nature des choses que la Totalisation permet d'atteindre.

> *Accepter que le logiciel soit vivant, complexe et jamais terminé, c'est déjà faire la moitié du chemin vers l'excellence. L'autre moitié, c'est décider de l'honorer pleinement malgré - et grâce à - tout ce qu'il est.*

Chapitre 3

Les fractures dans la relation avec le produit logiciel

Pour se réconcilier, il faut d'abord reconnaître les blessures. Ce chapitre cartographie les fractures profondes qui traversent la relation de l'industrie avec son propre produit - des fractures qui expliquent pourquoi tant de logiciels déçoivent, même ceux créés avec de bonnes intentions.

La fracture entre développeurs et utilisateurs

Il existe, dans l'industrie du logiciel, une fracture culturelle profonde entre ceux qui créent le produit et ceux qui l'utilisent. Cette fracture n'est pas nouvelle, elle n'est pas universelle, et elle n'est pas intentionnelle. Mais elle est réelle, persistante, et elle fait des ravages énormes dans la qualité des produits qui en résultent.

D'un côté de cette fracture, on trouve les développeurs. Ils pensent en systèmes, en architectures, en patterns. Leur monde est celui de la précision, de la logique, de la cohérence formelle. Ils voient le produit de l'intérieur : ses composantes, ses dépendances, ses possibilités techniques. Ils ont tendance à valoriser l'élégance technique, la puissance des fonctionnalités, la richesse des options offertes.

De l'autre côté, on trouve les utilisateurs. Ils pensent en tâches, en résultats, en émotions. Leur monde est celui de l'objectif à atteindre, du problème à résoudre, de la frustration à éviter. Ils voient le produit de l'extérieur : son interface, sa rapidité, sa capacité à s'intégrer dans leur flux de travail. Ils valorisent la simplicité, la fiabilité, la compatibilité avec leur façon de penser et de travailler.

Ces deux visions ne sont pas incompatibles. Mais elles sont rarement réconciliées. Trop souvent, on crée des produits qui font la joie des développeurs et la frustration des utilisateurs. Des interfaces surchargées de fonctionnalités dont personne n'a besoin. Des workflows qui réflètent la logique de la base de données plutôt que la logique du travail humain. Des systèmes « puissants » que personne ne sait utiliser sans formation intensive.

Totalisation répond à cette fracture en affirmant que le produit logiciel n'appartient ni aux développeurs ni aux utilisateurs : il existe *dans la relation* entre eux. Il est le pont entre deux mondes. Et un pont qui ne tient que d'un

côté n'est pas un pont - c'est une impasse. Construire ce pont exige qu'on comprenne les deux rives avec égale profondeur et qu'on refuse de privilégier l'une au détriment de l'autre.

La fracture entre vision et réalité du produit

Chaque produit logiciel commence par une vision. Une idée de ce qu'il sera, de ce qu'il fera, de la valeur qu'il créera. Cette vision est souvent puissante, parfois même belle. Elle mobilise des équipes, attire des investissements, génère de l'enthousiasme. Et puis vient la réalité du développement - avec ses contraintes techniques, ses délais compressés, ses découvertes désagréables en cours de route, ses compromis à faire sur les fonctionnalités, la performance, l'ergonomie.

À chaque compromis, à chaque « on fera ça dans la prochaine version », à chaque fonctionnalité coupée pour respecter un délai, la distance entre la vision originale et le produit réel grandit un peu. Et quand le produit est finalement livré, il ressemble rarement à ce qu'on avait imaginé. Pas nécessairement parce qu'il est mauvais - parfois, le produit réel est meilleur que la vision initiale parce qu'il a été raffiné par le contact avec la réalité. Mais souvent, cette distance est source de désillusion, de sentiment d'échec, d'un malaise diffus que personne ne sait vraiment nommer.

La fracture entre vision et réalité n'est pas inévitable. Elle est le résultat d'une vision mal ancrée dans la réalité dès le départ, ou d'une réalité mal gérée qui dévie de la vision sans qu'on s'en rende compte. Totalisation propose un principe pour résoudre cette fracture : *la vision doit être totale dès le début et la réalité doit être confrontée à cette vision à chaque étape, sans exception.* Pas pour maintenir une vision figée - les visions doivent évoluer. Mais pour

s'assurer que les écarts entre vision et réalité sont toujours conscients, toujours délibérés, toujours compris.

La fracture entre qualité perçue et qualité réelle

On a déjà évoqué l'invisibilité de la qualité logicielle. Cette invisibilité crée une fracture spécifique entre ce que les parties prenantes perçoivent comme de la qualité et ce qu'est réellement la qualité du produit. Cette fracture est particulièrement pernicieuse parce qu'elle peut persister longtemps sans être détectée - et parce que quand elle se révèle, c'est généralement au pire moment possible.

Un produit avec une belle interface peut cacher une architecture instable. Un produit qui semble performer peut reposer sur des hacks qui le rendront non maintenable dans six mois. Un produit qui satisfait les utilisateurs actuels peut être totalement incapable d'absorber la croissance prévue. La qualité perçue est réelle dans l'expérience immédiate - et trompeuse dans le temps. La qualité réelle, elle, se manifeste dans la durée : dans la capacité du produit à évoluer, à résister aux pannes, à s'adapter aux changements de contexte sans s'effondrer.

Totalisation répond à cette fracture en établissant un principe fondamental : *la qualité doit être totale ou elle ne l'est pas.* Un produit de qualité au sens ŢŢĻŞŢ~ est un produit dont la qualité est cohérente à tous les niveaux - apparente et profonde, immédiate et durable, visible et invisible. Toute fracture entre ces niveaux est une dette de qualité qui devra être remboursée, avec intérêts, dans le futur.

La fracture temporelle : ce qu'on construit vs ce qui durera

L'industrie du logiciel vit dans un temps court. Les sprints durent deux semaines. Les roadmaps s'étendent sur un trimestre, parfois une année. Les visions de long terme sont nobles en théorie, mais elles cèdent souvent au premier problème urgent qui se présente. Dans ce contexte de temporalité compressée, on construit pour le présent - pour la démo de vendredi, pour la conférence du mois prochain, pour les chiffres du trimestre. On ne construit pas pour ce qui durera.

La fracture temporelle, c'est l'écart entre les décisions qu'on prend aujourd'hui et leurs conséquences à long terme. C'est l'architecture qui semble raisonnable maintenant mais qui s'avérera un goulot d'étranglement dans deux ans. C'est le choix technologique qui semble moderne aujourd'hui et qui sera obsolète demain. C'est la dette technique qu'on accumule sprint après sprint en sachant qu'on la paiera un jour, mais en espoir que ce sera après qu'on aura passé à autre chose.

Totalisation repose sur une vision temporelle élargie. « Avant, pendant et après - en continu, sans interruption » n'est pas une métaphore de process. C'est une posture face au temps. C'est la décision de toujours voir le produit dans sa totalité temporelle, y compris les conséquences futures de ce qu'on décide maintenant. C'est la discipline de ne pas sacrifier le long terme sur l'autel de l'urgence court-termiste, sauf quand ce sacrifice est conscient, déliberé, et documenté comme tel.

Comment ces fractures génèrent des logiciels décevants

Ces quatre fractures ne sont pas indépendantes. Elles s'alimentent mutuellement dans une dynamique qui expliquer pourquoi tant de projets

logiciels, même bien intentéonnés, même dotés de bonnes équipes, finissent par décevoir. Quand on ne comprend pas l'utilisateur, la vision dérive. Quand la vision dérive, la qualité perçue et la qualité réelle se découplent. Quand ces qualités se découplent, on prend des décisions à court terme qui fracturent la temporalité. Et la boucle recommence, chaque fois un peu plus profondément.

Un logiciel décevant n'est pas nécessairement un logiciel mal codé. C'est un logiciel où l'une ou plusieurs de ces fractures n'ont pas été reconnues, adressées et réconciliées. C'est un produit qui a été construit en morceaux - un morceau par équipe, un morceau par sprint, un morceau par trimestre - sans jamais être vu, pensé et réalisé comme un tout cohérent.

La réponse de ŢŢĻŞŢ~ à ces fractures n'est pas de les éliminer une par une avec des outils spécifiques. C'est de cultiver un état d'esprit qui les rend visibles avant qu'elles ne se creusent - un état d'esprit de Totalisation qui refuse les angles morts, qui exige la cohérence, qui maintient la vision du tout même quand on travaille sur les parties.

ŢŢĻŞŢ~ ET LA MÉTHODE BEASTEST

Dans le cadre BEASTEST, les fractures décrites dans ce chapitre correspondent à des glissements entre niveaux. Un produit au niveau ROUGE est entrement défini par la fracture entre vision et réalité - il survit, il fonctionne à peine, mais personne ne comprend vraiment pourquoi ni pour qui. Un produit au niveau VERT a commencé à recoudre certaines fractures - on comprend ses utilisateurs, on a une vision stable - mais la fracture temporelle et la fracture de qualité persistent souvent. Le VIOLET, lui, est l'état où les quatre fractures sont reconnues, adressées et maintenues en conscience permanente - pas nécessairement éliminées,

mais totalisées.

Chapitre 4

ŢŢĻŞŢ~ face au produit : une nouvelle ontologie

Ce chapitre est le coeur philosophique de ce livre. Il présente la nouvelle façon de voir le produit logiciel que propose Totalisation - une ontologie qui réconcilie ses dimensions en les voyant non pas en séparation, mais dans leur totalité cohérente.

Voir le produit logiciel comme un tout indivisible

L'ontologie est la branche de la philosophie qui s'intéresse à la question de l'être : qu'est-ce qui existe, comment les choses existent-elles, quelle est la nature fondamentale de la réalité ? Appliquer une réflexion ontologique au produit logiciel, c'est cesser de le voir comme une liste de fonctionnalités ou un artefact technique pour le voir dans ce qu'il *est* fondamentalement.

L'ontologie de Totalisation commence par un postulat simple mais radical : *le produit logiciel est un tout indivisible.* Non pas parce qu'on ne peut pas le décomposer en parties - bien sûr qu'on le peut, et c'est nécessaire pour le construire. Mais parce que ces parties n'existent que dans leur relation au tout. Une fonctionnalité isolée n'a pas de sens. Un module indépendant de son contexte n'a pas de valeur. Un composant coupé de la vision qui l'a généré est un artefact sans âme.

Voir le produit comme un tout indivisible ne signifie pas qu'on ne peut pas travailler sur des parties. Cela signifie qu'en travaillant sur une partie, on garde toujours conscience du tout. Qu'on ne décide jamais de quelque chose dans la partie sans se demander comment cela affecte le tout. Qu'on ne livre jamais une fonctionnalité sans se demander comment elle s'intègre dans la cohérence globale du produit.

Cette vision holistique a une conséquence pratique profonde : elle change la manière dont on évalue le succès. On ne réussit pas parce qu'une fonctionnalité est livrée. On réussit quand le tout est avancé d'une manière cohérente. On échoue non pas quand une fonctionnalité est boguée, mais quand la cohérence du tout est compromise. C'est une mesure du succès plus exigeante - et infiniment plus juste.

La réconciliation avec les exigences : elles font partie du produit

Dans la relation traditionnelle avec le produit logiciel, les exigences sont souvent vécues comme une contrainte extérieure. Elles arrivent « d'en haut » - des clients, des utilisateurs, des gestionnaires - et les développeurs doivent les implémenter, parfois en les comprenant à peine, parfois en estimant qu'elles sont mal formulées ou techniquement naïves. Cette relation de tension avec les exigences est une source majeure de fracture dans le développement logiciel.

Totalisation propose une réconciliation profonde avec les exigences en changeant radicalement la façon dont on les perçoit. Les exigences ne sont pas extérieures au produit. *Elles font partie du produit.* Elles en sont la voix - la voix de ce que le produit doit être, de ce qu'il doit faire, de la valeur qu'il doit

créer. Ignorer les exigences, les mépriser, les contourner, c'est ignorer une partie essentielle de l'identité du produit.

Cette réconciliation demande un changement d'attitude profond. Elle demande de cesser de voir les exigences comme un document PDF à implémenter et de commencer à les voir comme une conversation permanente avec la réalité que le produit cherche à servir. Elle demande d'accepter que les exigences soient imparfaites, changeantes, parfois contradictoires - non pas parce que ceux qui les formulent sont incompétents, mais parce que la réalité qu'elles cherchent à capturer est elle-même imparfaite, changeante et parfois contradictoire.

Se réconcilier avec les exigences, c'est aussi accepter d'être surpris par elles. D'apprendre d'elles. De les laisser remettre en question des décisions techniques qu'on croyait acquises. C'est accepter que le produit soit co-construit entre ceux qui le font et ceux qui l'utilisent - et que les exigences soient l'interface de cette co-construction.

La réconciliation avec le code : il est le produit, pas seulement son conteneur

Il existe dans l'industrie du logiciel un rapport étrange au code. D'un côté, les développeurs y attachent une grande importance - ils débattent de style, d'architecture, d'élégance. De l'autre, dans la culture de management de produit, le code est souvent perçu comme un moyen, pas comme une fin. Comme un conteneur qui permet de « livrer » des fonctionnalités, mais pas comme quelque chose qui a une valeur en soi.

Totalisation refuse cette dichotomie. Le code n'est pas le conteneur du produit. *Le code est le produit.* Il est la matière même dont le produit est fait, la

substance dans laquelle toutes les décisions - architecturales, fonctionnelles, qualitatives - sont cristallisées. Un code bien écrit n'est pas seulement plus agréable à lire : il est un produit meilleur. Un code lisible, maintenable, testable est un produit qui a de l'avenir. Un code chaotique, non testé, mal documenté est un produit condamné à se dégrader.

Se réconcilier avec le code, c'est lui accorder la dignité qui lui revient. C'est refuser le syndrome du « fera la job pour l'instant » quand on sait qu'il ne fera pas la job dans six mois. C'est investir dans la lisibilité, la structure, les tests, non pas comme des luxes qu'on s'accorde quand on a du temps, mais comme des constituantes essentielles du produit qu'on est en train de créer. C'est comprendre que la qualité du code est la qualité du produit - pas une dimension séparée, pas un « extra », mais le produit lui-même dans sa réalité la plus concrète.

Dans l'échelle BEASTEST, la réconciliation avec le code marque un saut qualitatif significatif. On passe du niveau ORANGE - où le code est un moyen de survivre, de livrer, de faire taire les urgences - au niveau VERT - où le code est reconnu comme la matière du produit et traité avec l'attention et le soin qu'il mérite. Et c'est un saut qui ne se fait pas par plus de discipline : il se fait par plus de compréhension. Par la Totalisation du regard qu'on pose sur ce qu'on crée.

La réconciliation avec les tests : ils révèlent le produit tel qu'il est vraiment

Les tests sont peut-être la dimension du produit logiciel avec laquelle la relation est la plus complexe, la plus ambivalente, la plus chargée de non-dits. Dans beaucoup d'équipes, les tests sont vécus comme une obligation - nécessaires, oui, mais pénibles. Comme une double-vérification du travail déjà

fait, une activité qui ralentit le rythme de livraison, une exigence externe plutôt qu'une pratique interne créatrice de valeur.

Totalisation renverse complètement cette perspective. Les tests ne vérifient pas le produit. *Ils le révèlent.* Ils sont le miroir le plus honnte que le produit puisse avoir. Chaque test échouant n'est pas un échec de l'ingénieur - c'est une révélation de la réalité. Une information précieuse sur ce que le produit est vraiment, par opposition à ce qu'on croyait qu'il était. Cette information a une valeur considérable - bien plus grande que la valeur illusoire d'un logiciel qu'on croit fonctionner parce qu'on n'a pas pris le temps de le tester.

Se réconcilier avec les tests, c'est désirer cette révélation plutôt que de la craindre. C'est accueillir chaque test qui échoue comme un cadeau - l'information qu'on avait besoin d'avoir avant la mise en production, avant l'utilisateur, avant la crise. C'est comprendre que tester abondamment, c'est voir son produit avec plus de clarté, plus d'honnêteté, plus de Totalisation.

Les tests sont également une expression de respect envers les utilisateurs. En testant, on dit : « On a vérifié que ce qu'on te donne fonctionne réellement. On a pris le temps de le savoir avant que tu ne le découvres toi-même. » Cette posture de respect est profondément alignée avec l'esprit de Totalisation - qui refuse de négliger l'*après* au profit du *pendant*, et qui reconnatît que la livraison n'est pas une fin mais une continuité.

La réconciliation avec la documentation : mémoire vivante du produit

La documentation est probablement la parente pauvre du produit logiciel. Elle est souvent considérée comme un devoir bureaucratique, un prérequis imposé plutôt qu'une création de valeur. Les développeurs la négligent parce qu'ils

préfèrent écrire du code. Les équipes la reportent parce qu'elles n'ont « pas le temps ». Et quand elle existe enfin, elle est souvent déjà obsolète parce que le produit a évolué plus vite qu'elle n'a pu être écrite.

Totalisation transforme radicalement la relation à la documentation en la définissant pour ce qu'elle est vraiment : *la mémoire vivante du produit.* Non pas un registre mort de ce que le produit faisait à un moment donné, mais la trace continue de ses intentions, de ses décisions, de son évolution. Une documentation vivante dit non seulement *comment* le produit fonctionne, mais *pourquoi* il fonctionne de cette façon. Elle capture les raisonnements, pas seulement les résultats. Elle préserve les décisions d'architecture avec leurs contextes et leurs compromis. Elle évite que la même erreur soit faite deux fois parce que personne ne se rappelle pourquoi une décision différente avait été prise il y a dix-huit mois.

La réconciliation avec la documentation est aussi une réconciliation avec le temps. Documenter, c'est reconnaître qu'on n'est pas la dernière personne qui touchera ce produit. C'est faire preuve de générosité envers ceux qui viendront après - qu'ils fassent partie de notre équipe ou non. C'est transformer le produit logiciel, qui est par nature fugace et évolutif, en quelque chose qui a une histoire, une mémoire, une continuité. C'est honorer le *avant* et préparer l'*après* pendant le *pendant*. C'est, en un mot, de la Totalisation.

> *La documentation n'est pas la trace du produit. Elle est une partie intégrante du produit. Un produit sans documentation est un produit qui commence à mourir dès qu'une personne qui le connatît quitte l'équipe.*

Le produit totalisé : vers une vision entière

La nouvelle ontologie que Totalisation propose pour le produit logiciel peut se résumer ainsi : un produit logiciel est une réalité totale, vivante et relationnelle, qui existe dans la continuité du temps, et dont la qualité est une propriété du tout plutôt que la somme des qualités de ses parties. Cette définition réconcilie les exigences, le code, les tests et la documentation non pas comme des activités distinctes, mais comme des dimensions d'une même réalité - le produit dans sa totalité.

Cette vision entière du produit n'est pas une vision naïve. Elle ne prétend pas qu'on peut tout voir, tout prévoir, tout contrôler. Au contraire. Elle accepte l'incertitude, le changement, l'imperfection - mais elle refuse que ces réalités soient des excuses pour la fragmentation, pour la négligence, pour les angles morts qui résultent de ne voir qu'une partie de ce qu'on crée.

Un produit totalisé n'est pas nécessairement un produit parfait. C'est un produit vu dans toute sa réalité, compris dans toutes ses dimensions, créé avec une conscience permanente de sa totalité. C'est un produit dont les créateurs refusent de se fermer les yeux sur ses parties inconfortables - la dette technique, les exigences mal comprises, les tests insuffisants, la documentation absente - et qui font de cette conscience ouverte le fondement de leur excellence.

C'est ça, la réconciliation que T̰T̰L̰S̰T̰~ propose. Pas une réconciliation facile, pas une réconciliation sentimentale. Une réconciliation lucide, rigoureuse, exigeante - qui commence par voir les choses telles qu'elles sont et qui refuse de s'arrêter avant d'avoir vu le tout.

Dimension du produit	Vision traditionnelle	Vision ŢŢĻṢŢ~
Les exigences	Contrainte extérieure à implémenter	Voix du produit - partie intégrante de son identité
Le code	Conteneur technique du produit	La matière même dont le produit est fait
Les tests	Vérification pénible et obligatoire	Miroir honnte révélant le produit tel qu'il est
La documentation	Devoir bureaucratique, souvent négligé	Mémoire vivante et continuité du produit
La qualité	Propriété de la surface visible	Cohérence totale entre apparence et profondeur
Le temps	Sprint actuel, roadmap court terme	Continuum avant-pendant-après, sans interruption

Note de fin de Partie A

« Ce n'est pas la fin du voyage. C'est la fin du premier acte.

Et le deuxième acte commence exactement là où celui-ci se termine - dans la conscience de ce que le produit est vraiment,

et dans le courage de le construire en conséquence. »

Tu arrives à la fin de la Partie A de *ŢŢĻṢŢ~ - Réconciliation avec le produit logiciel*. Tu as traversé le paradoxe fondateur de l'industrie - celle qui construit des logiciels sans toujours les comprendre vraiment. Tu as exploré la nature unique de ce produit étrange qu'est le logiciel : immatériel, complexe, vivant, jamais terminé. Tu as cartographié les fractures profondes qui traversent la relation de l'industrie avec son propre produit. Et tu as découvert la nouvelle

ontologie que Totalisation propose : voir le produit comme un tout indivisible, se réconcilier avec toutes ses dimensions - exigences, code, tests, documentation - et refuser les angles morts qui mènent à des logiciels décevants.

Cette Partie A a posé les fondations philosophiques. Elle a défini les termes, nommé les réalités, établi les principes. Mais la philosophie sans la pratique est incomplète. Et Totalisation, par définition, est complète ou elle n'est pas.

Aperçu de la Partie B

T̊T̊L̊S̊T~ - Réconciliation avec le produit logiciel - Partie B

Vivre la réconciliation : le produit totalisé en action

La Partie B entrera dans le vif de la pratique philosophique. Elle explorera comment la réconciliation avec le produit logiciel se vit au quotidien - dans les conversations difficiles sur les exigences, dans les revues de code, dans les rétrospectives d'équipe, dans les décisions d'architecture. Elle examinera les catégories de produits logiciels à travers la lens de Totalisation, et présentera les conditions qui permettent à un produit d'atteindre les niveaux BLEU, INDIGO et VIOLET de la méthode BEASTEST. Elle proposera enfin une vision du collectif qui construit le produit - car un produit totalisé ne peut natre que d'un collectif qui partage la même philosophie de la complétude.

D'ici là, porte avec toi l'essentiel de ce que la Partie A a voulu te transmettre : le produit logiciel mérite d'être vu dans sa totalité. Il mérite qu'on cesse de le maltraiter par ignorance ou par précipitation. Il mérite d'être créé avec la

pleine conscience de ce qu'il est, de ce qu'il peut devenir et de la valeur réelle qu'il peut créer pour ceux qui l'utilisent.

La réconciliation a commencé. Elle se poursuit en toi, à chaque décision que tu prends en ayant vu le tout.

ŢŢĻŞŢ~

TOTALISATION

ŢŢĻŞŢ~ - Réconciliation avec le produit logiciel - Partie A

Auteur : Léo Michel Bélanger
Images et couverture : Soléane Bélanger
Publié par ORIGINALIUM | ŢŢĻŞŢ~

© Léo Michel Bélanger - Tous droits réservés
Montréal, Québec, Canada

Série ORIGINALIUM - ŢŢĻŞŢ~
Ţ=TO Ţ=TA Ļ=LI Ş=SA Ţ=TI ~=ON